Day 157 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 23, Day 4 reference

Watch: Maciej Kula - Hybrid Recommender Systems in Python (PyData) | Hybrid Recommender Systems (eggie545)

`day157_hybrid_recommenders.py`, segment by segment, in the order `main()` uses things. The parts (stack_fit, stack_predict, fw_design, switch_predict, hybrid_mf, hybrid_predict) are copied unchanged into `day157_exercise.py`; Days 154–156's parts are imported from their exercise files. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: P = predictions, one column per model · w = blend weights, intercept first · n = training ratings of each film · X = film content (27 flags) · Z = content to factors · a = content bias

Segment 0 — Setup

Imports: three earlier days' parts, and their settings

CODE

```
HERE = os.path.dirname(os.path.abspath(__file__))
sys.path.insert(0, HERE)
with contextlib.redirect_stdout(io.StringIO()):
    import day154_exercise as d154
    import day155_exercise as d155
    import day156_exercise as d156
LU, LI = 5, 2
ITEM = dict(k=40, shrink=800)
ALS = dict(k=100, lam=0.125)
PROF = dict(shared=1, lam=30)
NAMES = ("bias model", "item-based", "ALS factors", "content profiles", "bias + content")
THRESHOLDS = (0, 1, 3, 5, 10, 20, 50, 100)
EPOCHS = 120
```

- `with contextlib.redirect_stdout(io.StringIO()):` — all three imports sit inside one with block, so none of their hand checks print.
- `ALS = dict(k=100, lam=0.125)` — the settings each earlier day chose on validation are reused, not re-tuned.
- `NAMES = (...)` — a tuple of names, in the same order as the prediction columns.

Segment 1 — The parts

stack_fit, stack_predict: the weighted hybrid

MATH

$$w = \operatorname{argmin} |y - [1, P]\, w|^2$$

CODE

```
def stack_fit(P, y):
    A = np.c_[np.ones(len(y)), P]
    return np.linalg.lstsq(A, y, rcond=None)[0]
def stack_predict(P, w):
    return w[0] + P @ w[1:]
```

- `np.c_[np.ones(len(y)), P]` — a column of 1s in front, so the first weight is the intercept.
- `np.linalg.lstsq(A, y, rcond=None)[0]` — the least-squares solution; [0] keeps it (lstsq also returns residuals and rank).
- `P @ w[1:]` — each row's predictions times the model weights, summed: one blended number per rating.

fw_design and switch_predict

MATH

$$w_m(i) = a_m + c_m \log(1 + n_i)$$

CODE

```
def fw_design(P, n):
    g = np.log1p(n)[:, None]
    return np.c_[g, P, P * g]
def switch_predict(P_rich, P_poor, n, threshold):
    return np.where(n < threshold, P_poor, P_rich)
```

- `np.log1p(n)[:, None]` — log(1 + n), safe at n = 0, as a column so it multiplies every model's column.
- `np.c_[g, P, P * g]` — the slope of the intercept, the plain weights, and each weight's slope in g.
- `np.where(n < threshold, P_poor, P_rich)` — picks cell by cell: the content model where the film has few ratings.

hybrid_mf and hybrid_predict: content inside the factors

MATH

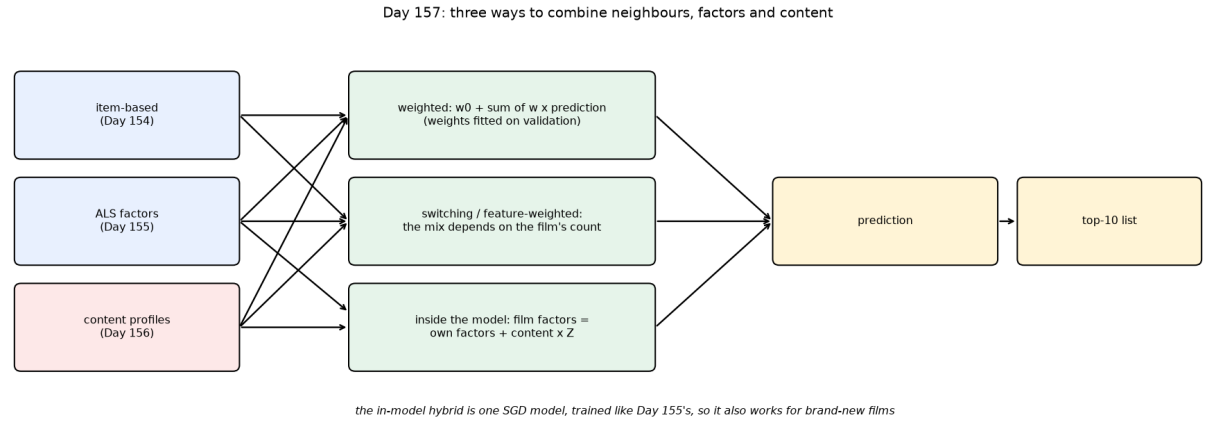
$$pred = \mu + b_u + b_i + x_i \cdot a + p_u \cdot (q_i + Z^T x_i)$$

CODE

```
def hybrid_mf(u, i, r, X, nu, ni, k, lam, lr=0.01, lr_s=0.1, lam_s=0.001, epochs=60, batch=128,
    seed=0, track=None):
    rng = np.random.RandomState(seed); f = X.shape[1]
    seen = np.bincount(i, minlength=ni) > 0
    F = dict(mu=r.mean(), P=0.1 * rng.randn(nu, k), Q=0.1 * rng.randn(ni, k) * seen[:, None],
             bu=np.zeros(nu), bi=np.zeros(ni), a=np.zeros(f), Z=0.1 * rng.randn(f, k) /
                 np.sqrt(f))
    hist = []
    for _ in range(epochs):
        o = rng.permutation(len(r))
        for s in range(0, len(o), batch):
            b = o[s:s + batch]; a, c = u[b], i[b]; xc = X[c]
            qe = F["Q"][c] + xc @ F["Z"]
            e = r[b] - (F["mu"] + F["bu"][a] + F["bi"][c] + xc @ F["a"] + (F["P"][a] * qe).sum(1))
            gp = e[:, None] * qe - lam * F["P"][a]
            gq = e[:, None] * F["P"][a] - lam * F["Q"][c]
            gz = xc.T @ (e[:, None] * F["P"][a]) / len(b) - lam_s * F["Z"]
            ga = xc.T @ e / len(b) - lam_s * F["a"]
            np.add.at(F["bu"], a, lr * (e - lam * F["bu"][a]))
            np.add.at(F["bi"], c, lr * (e - lam * F["bi"][c]))
            np.add.at(F["P"], a, lr * gp); np.add.at(F["Q"], c, lr * gq)
            F["Z"] += lr_s * gz; F["a"] += lr_s * ga
        hist.append(track(F) if track else ())
    return F, hist
def hybrid_predict(F, X, u, i):
    q = F["Q"][i] + X[i] @ F["Z"]
    return F["mu"] + F["bu"][u] + F["bi"][i] + X[i] @ F["a"] + (F["P"][u] * q).sum(1)
```

- `np.bincount(i, minlength=ni) > 0` — counts each film's training ratings; True where there is at least one.
- `0.1 * rng.randn(ni, k) * seen[:, None]` — random starting factors, multiplied by 0 for films with no ratings.
- `qe = F["Q"][c] + xc @ F["Z"]` — each film's own factors plus what its 27 content flags map to.
- `xc.T @ (e[:, None] * F["P"][a]) / len(b)` — the gradient for Z summed over the batch, then averaged: Z is shared by every film.
- `F["Z"] += lr_s * gz` — shared weights get their own learning rate; no np.add.at is needed, there is only one Z.

SYSTEM DIAGRAM



Three ways to combine recommenders.

Segment 2 — The experiments

ingredients: five models from three days' parts

CODE

```
F, _ = d155.als_mf(R, M, ALS["k"], ALS["lam"])
mu0, bu0, _ = d154.fit_biases(R, M, LU, 1e12); T = (R - mu0 - bu0[:, None]) * M
W = d156.fit_profiles(X, T, M, PROF["lam"], prior=d156.fit_shared_profile(X, T, M,
    PROF["shared"]))
Tb = (R - B) * M
Wb = d156.fit_profiles(X, Tb, M, PROF["lam"], prior=d156.fit_shared_profile(X, Tb, M,
    PROF["shared"]))
b = B[u[ev], i[ev]]
P = np.c_[b, b + c, d155.mf_predict(F, u[ev], i[ev]),
    mu0 + bu0[u[ev]] + d156.content_predict(W, X, u[ev], i[ev]), b + d156.content_predict(Wb, X,
        u[ev], i[ev])]
return P, M.sum(0)[i[ev]], dict(B=B, F=F, W=W, Wb=Wb, M=M, mu0=mu0, bu0=bu0)
```

- `d155.als_mf(R, M, ALS["k"], ALS["lam"])` — a dictionary's values passed by key: the same settings everywhere.
- `d154.fit_biases(R, M, LU, 1e12)` — a huge film penalty: mean + user bias only, Day 156's content-only base.
- `np.c_[b, b + c, ...]` — the five models' predictions side by side, one column each.
- `dict(B=B, F=F, ...)` — the fitted models are returned too, for the top-10 lists in section 5.

demo_blend: the checks

CODE

```
h = np.random.RandomState(1).permutation(len(va)); A, Bh = h[:len(h) // 2], h[len(h) // 2:]
for cols, nm in (((1, 2, 3), "three-model"), ((0, 1, 2, 3, 4), "five-model")):
    w = stack_fit(Pv[:, cols], r[va])
    rng = np.random.RandomState(0)
    wn = stack_fit(np.c_[Pv[:, cols], rng.randn(len(va))], r[va])
    en = score(stack_predict(np.c_[Pt[:, cols], rng.randn(len(te))], wn), r[te])
    wt = stack_fit(Pt[:, cols], r[te])
    el = score(stack_predict(Pt[:, cols], wt), r[te])
    ha = score(stack_predict(Pv[Bh][:, cols], stack_fit(Pv[A][:, cols], r[va][A])), r[va][Bh])
```

- `A, Bh = h[:len(h) // 2], h[len(h) // 2:]` — a random half of the validation ratings, and the other half.

- `for cols, nm in (((1, 2, 3), "three-model"), ...)` — a tuple of (columns, name) pairs, unpacked in the loop.
- `np.c_[Pv[:, cols], rng.randn(len(va))]` — the blend's inputs plus one column of random numbers: the noise control.
- `stack_fit(Pt[:, cols], r[te])` — the deliberate leak: weights fitted on the test ratings, printed only for comparison.
- `Pv[Bh][:, cols]` — rows first (the second half), then the columns of the blend.

demo_switch: the weights at four film counts

CODE

```
g = np.log1p(np.array([1, 10, 100, 400]))
ww = w[2:5][None, :] + np.outer(g, w[5:8])
```

- `w[2:5][None, :] + np.outer(g, w[5:8])` — plain weights as a row, plus each count times each weight's slope: one row per count.
- `np.outer(g, w[5:8])` — every pair of a count and a slope multiplied: a 4 x 3 table.

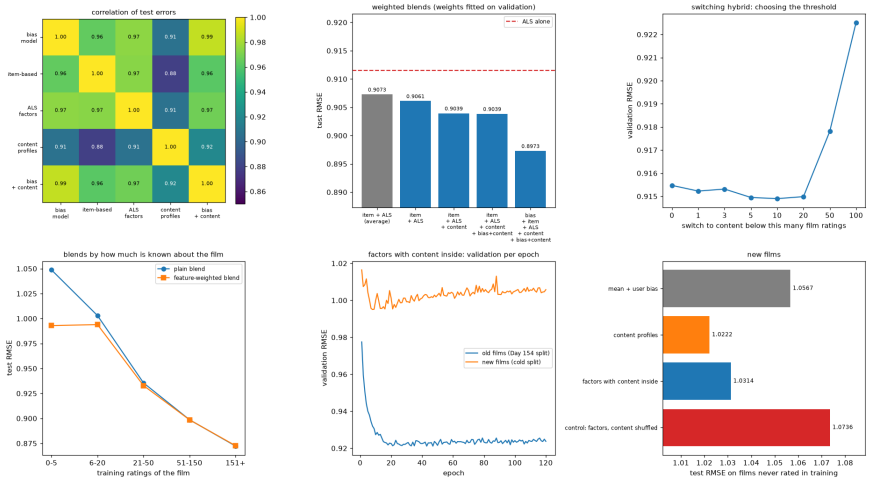
demo_hybrid_mf and demo_topn

CODE

```
vt = lambda F: (score(hybrid_predict(F, X, u[va], i[va]), r[va]),)
t0 = time.time(); F, h = hybrid_mf(u[fi], i[fi], r[fi], X, nu, ni, ALS["k"], ALS["lam"],
    epochs=EPOCHS, track=vt)
H = np.array([x[0] for x in h]); ep = int(np.argmin(H)) + 1
hyb = (Fh["mu"] + Fh["bu"][:, None] + Fh["bi"][None, :] + (X @ Fh["a"])[None, :]
       + Fh["P"] @ (Fh["Q"] + X @ Fh["Z"]).T)
```

- `lambda F: (score(...),)` — a one-item tuple, the record hybrid_mf appends after each epoch.
- `int(np.argmin(H)) + 1` — the epoch with the lowest validation error, counted from 1.
- `(X @ Fh["a"])[None, :]` — each film's content bias, as a row added to every user.
- `Fh["P"] @ (Fh["Q"] + X @ Fh["Z"]).T` — every user's factors against every film's combined factors: a full table of scores.

REAL GRAPH



Real graph from the script.

Quick Syntax Glossary — new or reinforced constructs

- `np.linalg.lstsq(A, y, rcond=None)[0]` — least-squares weights.
- `np.log1p(n)` — log(1 + n), safe at 0.
- `np.where(cond, a, b)` — choose cell by cell.
- `np.bincount(i, minlength=n)` — count each index.
- `np.outer(a, b)` — every pair multiplied.
- `A[rows][:, cols]` — rows first, then columns.
- `lambda F: (...,)` — a small unnamed function returning a tuple.
- `dict(k=100, lam=0.125)` — settings kept together by name.

End of Day 157 syntax reference. For the experiments and what they showed, see the Day 157 complete guide.